

1

1.2

We minimize the variational bound on the negative LL because the LL itself requires integrating over the denoising trajectory and is thus intractable.

$$\begin{aligned}
-\log p_\theta(x_0) &= -\log \int p_\theta(x_{0:T}) dx_{1:T} && \text{(marginalization)} \\
&= -\log \int q(x_{1:T}|x_0) \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} dx_{1:T} \\
&= -\log \mathbb{E}_{x_{1:T} \sim q(\cdot|x_0)} \left[\frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right] && \text{(definition)} \\
&\leq \mathbb{E}_{x_{1:T} \sim q(\cdot|x_0)} \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right] && \text{(Jensen, } -\log \text{ convex)} \\
\mathbb{E}_{x_0 \sim q} [-\log p_\theta(x_0)] &\leq \mathbb{E}_{x_0 \sim q, x_{1:T} \sim q(\cdot|x_0)} \left[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)} \right] \\
&= \mathbb{E}_q \left[-\log \frac{p(x_T) \prod_{t \geq 1} p_\theta(x_{t-1}|x_t)}{\prod_{t \geq 1} q(x_t|x_{t-1})} \right] && \text{(MC)} \\
&= \mathbb{E}_q \left[-\log p(x_T) - \sum_{t \geq 1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})} \right] \\
&=: L
\end{aligned}$$

1.3

(18) is justified above.

(20):

$$\begin{aligned}
\frac{q(x_{t-1}|x_0)}{q(x_{t-1}|x_t, x_0) \cdot q(x_t|x_0)} &= \frac{q(x_{t-1}, x_0)}{q(x_{t-1}|x_t, x_0) \cdot q(x_t, x_0)} \text{(Multiply by } q(x_0)) \\
&= \frac{q(x_{t-1}, x_0)}{q(x_{t-1}, x_t, x_0)} && \text{(Bayes' rule)} \\
&= \frac{1}{q(x_t|x_{t-1}, x_0)} && \text{(Bayes' rule)} \\
&= \frac{1}{q(x_t|x_{t-1})} && \text{(Markov)}
\end{aligned}$$

(21):

$$\begin{aligned}
\sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \cdot \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} &= \sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \sum_{t>1} \log \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
&= \sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \log \prod_{t>1} \frac{q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
&= \log \frac{q(x_1|x_0)}{q(x_T|x_0)} + \sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} \\
&= \log \frac{1}{q(x_T|x_0)} + \sum_{t>1} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)} + \log q(x_1|x_0)
\end{aligned}$$

Combining the dangling terms gives the result.

(22): We invoke the identity $\mathbb{E}_{X,Y}[f(X,Y)] = \mathbb{E}_X[\mathbb{E}_{Y|X}[f(X,Y)]]$. Each individual log term can be transformed in the same way with this trick. Using the first term as an example:

$$\begin{aligned}
\mathbb{E}_q \left[-\log \frac{p(x_T)}{q(x_T|x_0)} \right] &= \mathbb{E}_{x_0:T \sim q} \left[-\log \frac{p(x_T)}{q(x_T|x_0)} \right] \\
&= \mathbb{E}_{x_0 \sim q} \mathbb{E}_{x_1:T \sim q(\cdot|x_0)} \left[-\log \frac{p(x_T)}{q(x_T|x_0)} \right] \\
&= \mathbb{E}_{x_0 \sim q} [D_{\text{KL}}(q(x_T|x_0) \parallel p(x_T))] \\
&= \mathbb{E}_q [D_{\text{KL}}(q(x_T|x_0) \parallel p(x_T))]
\end{aligned}$$

1.4

We show this with induction. The $t = 1$ case is trivial so we omit it. Now assume $q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}; \sqrt{\alpha_{t-1}}x_0, (1 - \alpha_{t-1})I)$. The relationship between x_t, x_{t-1} is equivalent to

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\varepsilon$$

where $\varepsilon \sim \mathcal{N}(0, I)$. Since x_{t-1} is normally distributed, x_t is as well.

$$\begin{aligned}
\mathbb{E}[x_t] &= \sqrt{1 - \beta_t}\mathbb{E}[x_{t-1}] = \sqrt{1 - \beta_t}\sqrt{\alpha_{t-1}}x_0 = \sqrt{\alpha_t}x_0 \\
\mathbb{V}[x_t] &= \alpha_t\mathbb{V}[x_{t-1}] + \beta_t I = \alpha_t(1 - \alpha_{t-1})I + (1 - \alpha_t)I = (1 - \alpha_t)I
\end{aligned}$$

1.5

$$\begin{aligned}
&q(x_{t-1}|x_0, x_t) \\
&= \frac{q(x_t|x_0, x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
&= \frac{q(x_t|x_{t-1})q(x_{t-1}|x_0)}{q(x_t|x_0)} \\
&\propto (q(x_t|x_{t-1})q(x_{t-1}|x_0)) \\
&\propto \exp\left(-\frac{1}{2\beta_t} \|x_t - \sqrt{1 - \beta_t}x_{t-1}\|^2\right) \exp\left(-\frac{1}{2(1 - \alpha_{t-1})} \|x_{t-1} - \sqrt{\alpha_{t-1}}x_0\|^2\right) \\
&\propto \exp\left(-\frac{1}{2\beta_t} (-2\sqrt{1 - \beta_t}x_t^T x_{t-1} + (1 - \beta_t) \|x_{t-1}\|^2) - \frac{1}{2(1 - \alpha_{t-1})} (\|x_{t-1}\|^2 - 2\sqrt{\alpha_{t-1}}x_{t-1}^T x_0)\right)
\end{aligned}$$

The term quadratic in $\|x_{t-1}\|^2$ has the coefficient

$$-\frac{1}{2} \left(\frac{1 - \beta_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}} \right)$$

Since the covariance matrix must have the form cI due to repeated Gaussian conditioning, we know that

$$c = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{(1 - \beta_t)(1 - \bar{\alpha}_{t-1}) + \beta_t} = \tilde{\beta}_t$$

Collecting the remaining terms gives $\tilde{\mu}_t(x_t, x_0)$.

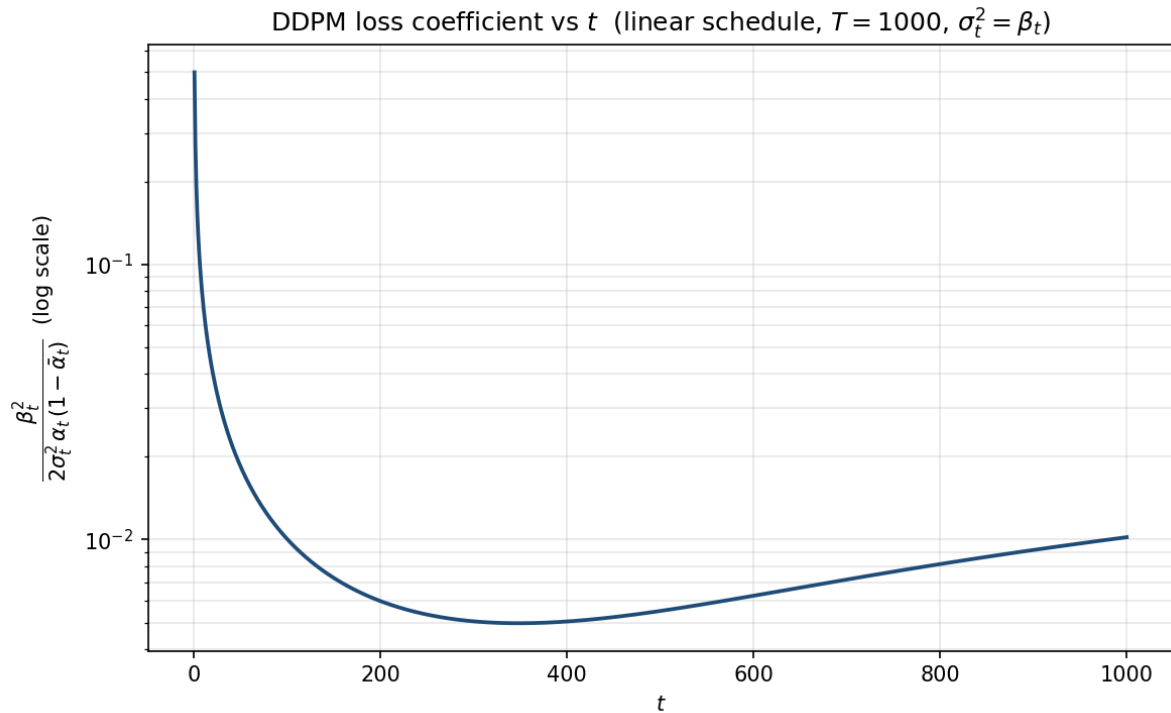
1.6

All the terms except the last do not depend on θ and therefore can be extracted into C . The quadratic form of a scaled identity matrix is just the scaled L_2 norm.

1.7

There is no nontrivial algebra in this step.

1.8



2

2.2

The term $\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)}$ is inaccessible to us through data because the empirical distribution of $p(x)$ is a collection of delta functions.

2.3

We need to show the cross term is equal to the trace.

$$\begin{aligned}
\mathbb{E}[s_\theta^T(x) \nabla_x \log p(x)] &= \mathbb{E}\left[\sum_i \frac{1}{p(x)} s_i(x) \partial_i p(x)\right] \\
&= \sum_i \int s_i(x) \partial_i p(x) dx && (\mathbb{E} \text{ absorbs } p(x)) \\
&= \sum_i \left(\int [s_i(x) p(x)]_{x_i=-\infty}^{x_i=\infty} dx_{-i} - \int \partial_i s_i(x) p(x) dx \right) && (\text{by parts in } x_i) \\
&= - \sum_i \int \partial_i s_i(x) p(x) dx && (p(x) \rightarrow 0 \text{ at } \infty) \\
&= - \sum_i \mathbb{E}[\partial_i s_i(x)] \\
&= - \text{tr}(\nabla_x s(x))
\end{aligned}$$

2.4

The trace of the Jacobian is computationally expensive.

2.5

$$\mathbb{E}[x|\tilde{x}] = \mathbb{E}[x - \tilde{x}|\tilde{x}] + \tilde{x}$$

We now show the conditional expectation on the RHS is equal to the σ^2 term. By definition of conditional expectation,

$$\mathbb{E}[x - \tilde{x}|\tilde{x}] = \int (x - \tilde{x}) p(x|\tilde{x}) dx$$

Moreover,

$$\begin{aligned}
\nabla \log p_\sigma(\tilde{x}) &= \frac{1}{p_\sigma(\tilde{x})} \nabla_{\tilde{x}} p_\sigma(\tilde{x}) \\
&= \frac{1}{p_\sigma(\tilde{x})} \nabla_{\tilde{x}} \left(\int p(\tilde{x}|x) p(x) dx \right) \\
&= \frac{1}{p_\sigma(\tilde{x})} \left(\int \nabla_{\tilde{x}} p(\tilde{x}|x) p(x) dx \right) \\
&= \frac{1}{p_\sigma(\tilde{x})} \left(\int \left(-\frac{\tilde{x} - x}{\sigma^2} \right) p(\tilde{x}|x) p(x) dx \right) \\
&= \frac{1}{p_\sigma(\tilde{x})} \left(\int \left(\frac{x - \tilde{x}}{\sigma^2} \right) p(\tilde{x}|x) p(x) dx \right) \\
&= \frac{1}{p_\sigma(\tilde{x})} \left(\int \left(\frac{x - \tilde{x}}{\sigma^2} \right) p(x|\tilde{x}) p_\sigma(\tilde{x}) dx \right) \\
&= \frac{1}{\sigma^2} \int (x - \tilde{x}) p(x|\tilde{x}) dx \\
&= \frac{1}{\sigma^2} (\mathbb{E}[x|\tilde{x}] - \tilde{x})
\end{aligned}$$

2.6

We would like to learn the term $\nabla \log p_\sigma(\tilde{x})$ so we minimize the MSE between $s_\theta(\tilde{x})$ and the desired quantity. The quadratic terms are either the same or do not depend on θ, σ . Therefore, let us match the cross terms.

$$\begin{aligned}
\mathbb{E}_p[s_\theta^T(x)\nabla_{\tilde{x}}\log p(\tilde{x}|x)] &= \int p(x)dx \int s_\theta^T(\tilde{x})\nabla_{\tilde{x}}\log p(\tilde{x}|x)p(\tilde{x}|x)d\tilde{x} \\
&= \int p(x)dx \int s_\theta^T(\tilde{x})\nabla_{\tilde{x}}p(\tilde{x}|x)d\tilde{x} \\
&= \int d\tilde{x}s_\theta^T(\tilde{x})\nabla_{\tilde{x}}\int p(x)p(\tilde{x}|x)dx \\
&= \int s_\theta^T(\tilde{x})\nabla_{\tilde{x}}p_\sigma(\tilde{x})d\tilde{x} \\
&= \int s_\theta^T(\tilde{x})p_\sigma(\tilde{x})\nabla_{\tilde{x}}\log p_\sigma(\tilde{x})d\tilde{x} \\
&= \mathbb{E}_{p_\sigma}[s_\theta(\tilde{x})^T\nabla_{\tilde{x}}\log p_\sigma(\tilde{x})]
\end{aligned}$$

2.7

The DSM objective aims to fit $\nabla\log p(\tilde{x}|x) = -\frac{\tilde{x}-x}{\sigma^2} = -\frac{\varepsilon}{\sigma}$ (since $\tilde{x} - x = \sigma\varepsilon$). The DDPM objective aims to fit ε , so $c(\sigma) = -\frac{1}{\sigma}$.

2.8

Take the discretization timestep to be η . Sample $x_0 \sim N(0, I)$. Repeat the following assignment:

$$\begin{aligned}
z_k &\sim N(0, I) \\
x_{k+1} &= x_k + \eta s_\theta(x_k) + \sqrt{2\eta}z_k
\end{aligned}$$

Return x_K . This works because s_θ replaces the unknown score function, and after $K \rightarrow \infty$ sampling steps $x_K \sim p_\sigma \approx p$ since p_σ is a stationary distribution.

3

3.2

The first is due to a Taylor series expansion since Δt is small, the second is due to $\beta(t)$ being a continuous function.

3.3

The general reverse-time formula for $dx = fdt + gdB_t$ is given by

$$dx = (f(x, t) - g(t)^2\nabla_x\log p_t(x))dt + g(t)d\bar{B}_t$$

Substituting,

$$dx = \left(-\frac{1}{2}\beta(t)x - \beta(t)\nabla_x\log p_t(x)\right)dt + \sqrt{\beta(t)}d\bar{B}_t$$

3.4

Step 1 (Euler–Maruyama discretization in reverse time). The reverse VP-SDE from Problem 3.3 is

$$dx = \left(-\frac{1}{2}\beta(t)x - \beta(t)\nabla_x\log p_t(x)\right)dt + \sqrt{\beta(t)}d\bar{B}_t.$$

We integrate this SDE from $t = 1$ down to $t = 0$, so a single Euler–Maruyama step advances reverse time by one increment, taking the state from index $i + 1$ (time $t + \Delta t$) to index i (time

t). Evaluating the drift and diffusion at the start of the step, x_{i+1} at time $t + \Delta t$, and using that for a backward step the elapsed (positive) increment is Δt (equivalently $dt = -\Delta t$):

$$x_i = x_{i+1} - \left(-\frac{1}{2}\beta(t + \Delta t)x_{i+1} - \beta(t + \Delta t)\nabla_x \log p_{t+\Delta t}(x_{i+1}) \right) \Delta t + \sqrt{\beta(t + \Delta t)}\sqrt{\Delta t}z_{i+1},$$

where $z_{i+1} \sim \mathcal{N}(0, I)$ (Euler–Maruyama: the drift scales with Δt , while the Brownian increment $d\bar{B}_t$ has standard deviation $\sqrt{\Delta t}$). The leading minus sign on the drift is because reverse time runs opposite to t , i.e. $dt < 0$ over the step while the step length $\Delta t > 0$; carrying the minus through flips both interior signs.

Now substitute the discrete schedule $\beta_{i+1} = \bar{\beta}_{i+1}\Delta t = \beta(t + \Delta t)\Delta t$ (Song21 App. B & D). Then $\beta(t + \Delta t)\Delta t = \beta_{i+1}$ and, for the diffusion term, $\sqrt{\beta(t + \Delta t)}\sqrt{\Delta t} = \sqrt{\beta(t + \Delta t)\Delta t} = \sqrt{\beta_{i+1}}$. Substituting:

$$x_i = x_{i+1} + \frac{1}{2}\beta_{i+1}x_{i+1} + \beta_{i+1}\nabla_x \log p_{t+\Delta t}(x_{i+1}) + \sqrt{\beta_{i+1}}z_{i+1}.$$

Step 2 (replace the true score by the learned model). The score $\nabla_x \log p_{t+\Delta t}(x_{i+1})$ is unknown, so we substitute the approximation $s_\theta(x_{i+1}, i + 1)$ (the learned network at state x_{i+1} and step $i + 1$):

$$x_i = x_{i+1} + \frac{1}{2}\beta_{i+1}x_{i+1} + \beta_{i+1}s_\theta(x_{i+1}, i + 1) + \sqrt{\beta_{i+1}}z_{i+1}.$$

Step 3 (collect terms). Group the two terms proportional to x_{i+1} , leaving the score and noise terms intact:

$$x_i = \left(1 + \frac{1}{2}\beta_{i+1} \right) x_{i+1} + \beta_{i+1}s_\theta(x_{i+1}, i + 1) + \sqrt{\beta_{i+1}}z_{i+1}.$$

The scalar coefficient of x_{i+1} is $1 + \frac{1}{2}\beta_{i+1}$: the $+1$ is the identity carried over from x_{i+1} itself, and the $+\frac{1}{2}\beta_{i+1}$ is the contribution of the $-\frac{1}{2}\beta x$ drift over one reverse step (the drift's own minus cancels against the reverse-time minus).

Step 4 (first-order matching with the Taylor expansion of $\frac{1}{\sqrt{1-\beta_{i+1}}}$). We want the prefactor in the DDPM form $\frac{1}{\sqrt{1-\beta_{i+1}}}$. Taylor-expand about $\beta_{i+1} = 0$:

$$\frac{1}{\sqrt{1-\beta_{i+1}}} = (1 - \beta_{i+1})^{-\frac{1}{2}} = 1 + \frac{1}{2}\beta_{i+1} + \frac{3}{8}\beta_{i+1}^2 + O(\beta_{i+1}^3),$$

so

$$1 + \frac{1}{2}\beta_{i+1} = \frac{1}{\sqrt{1-\beta_{i+1}}} + O(\beta_{i+1}^2).$$

Thus, to first order in β_{i+1} , the coefficient of x_{i+1} equals $\frac{1}{\sqrt{1-\beta_{i+1}}}$. This is exactly how the $-\frac{1}{2}\beta x$ drift combines with the $\frac{1}{\sqrt{1-\beta}}$ factor: the linearization of $(1 - \beta)^{-\frac{1}{2}}$ reproduces the $1 + \frac{1}{2}\beta$ that the Euler step produced from the drift.

We make the same first-order substitution on the score term. Since $\beta_{i+1} = \frac{1}{\sqrt{1-\beta_{i+1}}} \cdot \beta_{i+1}(1 + O(\beta_{i+1}))$, replacing the bare coefficient β_{i+1} multiplying s_θ by $\frac{\beta_{i+1}}{\sqrt{1-\beta_{i+1}}}$ changes it only by $O(\beta_{i+1}^2)$. Pulling the common factor $\frac{1}{\sqrt{1-\beta_{i+1}}}$ out of the x_{i+1} and s_θ terms, and keeping the diffusion term as is (its own correction is higher order), gives

$$x_i = \frac{1}{\sqrt{1 - \beta_{i+1}}} (x_{i+1} + \beta_{i+1} s_\theta(x_{i+1}, i+1)) + \sqrt{\beta_{i+1}} z_{i+1} + O(\beta_{i+1}^2).$$

Dropping the $O(\beta_{i+1}^2)$ terms (valid in the small-step limit $\beta_{i+1} \rightarrow 0$, where $\Delta t \rightarrow 0$) yields the claimed DDPM-style reverse update rule:

$$x_i = \frac{1}{\sqrt{1 - \beta_{i+1}}} (x_{i+1} + \beta_{i+1} s_\theta(x_{i+1}, i+1)) + \sqrt{\beta_{i+1}} z_{i+1}. \quad \blacksquare$$

3.5

Recall the SDE-based update rule derived in Problem 3.4 (with $\alpha_{i+1} = 1 - \beta_{i+1}$):

$$x_i = \frac{1}{\sqrt{1 - \beta_{i+1}}} (x_{i+1} + \beta_{i+1} s_\theta(x_{i+1}, i+1)) + \sqrt{\beta_{i+1}} z_{i+1},$$

and DDPM Algorithm 2 (Ho et al.) with the choice $\sigma_t = \sqrt{\beta_t}$:

$$x_i = \frac{1}{\sqrt{\alpha_{i+1}}} \left(x_{i+1} - \frac{\beta_{i+1}}{\sqrt{1 - \bar{\alpha}_{i+1}}} \varepsilon_\theta(x_{i+1}, i+1) \right) + \sqrt{\beta_{i+1}} z_{i+1}.$$

Both have identical prefactors $1/\sqrt{1 - \beta_{i+1}} = 1/\sqrt{\alpha_{i+1}}$ and identical noise terms $\sqrt{\beta_{i+1}} z_{i+1}$. They differ **only** in the correction term: the SDE form carries $+\beta_{i+1} s_\theta$, while DDPM carries $-(\beta_{i+1}/\sqrt{1 - \bar{\alpha}_{i+1}}) \varepsilon_\theta$. We must reconcile these.

The two networks parametrize different objects. The SDE/Langevin viewpoint uses a **score** model $s_\theta(x, t) \approx \nabla_x \log p_t(x)$, whereas DDPM trains a **noise** predictor $\varepsilon_\theta(x, t) \approx \varepsilon$, the standard normal noise that was injected. These are not the same function; converting between them requires the factor established in Problem 2.7.

The conversion factor (Problem 2.7). In DSM, the perturbation kernel is $\tilde{x} = x + \sigma \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$, and the conditional score is

$$\nabla_{\tilde{x}} \log p_\sigma(\tilde{x} | x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{\varepsilon}{\sigma}$$

(differentiating the Gaussian kernel; cf. Problem 2.5). Since the optimal denoiser matches this target, the optimal score relates to the optimal noise predictor by the **fixed** scalar $c(\sigma) = -1/\sigma$:

$$s_\theta = c(\sigma) \varepsilon_\theta = -\frac{\varepsilon_\theta}{\sigma}.$$

Identifying σ in the DDPM context. By the marginal forward kernel (Problem 1.4),

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I) \Rightarrow x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon,$$

so the standard deviation of the **total** noise added to the clean image x_0 at time t is

$$\sigma = \sqrt{1 - \bar{\alpha}_t}.$$

(This is the “ σ ” the hint refers to: not the per-step β_t , but the cumulative noise level $\sqrt{1 - \bar{\alpha}_t}$.) Hence at time $i+1$,

$$s_\theta(x_{i+1}, i+1) = -\frac{\varepsilon_\theta(x_{i+1}, i+1)}{\sqrt{1 - \bar{\alpha}_{i+1}}}.$$

Substitution. Plugging this into the SDE update converts the score correction into a noise correction:

$$\beta_{i+1} s_\theta(x_{i+1}, i+1) = \beta_{i+1} \cdot \left(-\frac{\varepsilon_\theta(x_{i+1}, i+1)}{\sqrt{1 - \bar{\alpha}_{i+1}}} \right) = -\frac{\beta_{i+1}}{\sqrt{1 - \bar{\alpha}_{i+1}}} \varepsilon_\theta(x_{i+1}, i+1).$$

Therefore the SDE update becomes

$$x_i = \frac{1}{\sqrt{1 - \beta_{i+1}}} \left(x_{i+1} - \frac{\beta_{i+1}}{\sqrt{1 - \bar{\alpha}_{i+1}}} \varepsilon_\theta(x_{i+1}, i+1) \right) + \sqrt{\beta_{i+1}} z_{i+1},$$

which is **exactly** DDPM Algorithm 2 with $\sigma_{i+1} = \sqrt{\beta_{i+1}}$.

Conclusion. The bare coefficient β_{i+1} in front of the score s_θ is replaced, after the score $\leftrightarrow$ noise change of variables, by $\beta_{i+1}/\sqrt{1 - \bar{\alpha}_{i+1}}$ in front of ε_θ . The “missing” multiplicative factor is precisely $1/\sigma = 1/\sqrt{1 - \bar{\alpha}_{i+1}}$, the score-to-noise conversion constant $c(\sigma) = -1/\sigma$ from Problem 2.7 (the sign being absorbed into the score’s $-\varepsilon_\theta$). The two algorithms are thus the same update written in two equivalent parametrizations of the same learned quantity.

3.6

(a) Plain Langevin dynamics is run at a **single fixed** noise level: it iterates $x_{k+1} = x_k + \eta s_\theta(x_k) + \sqrt{2\eta} z_k$ using only the score $\nabla_x \log p$ (or $\nabla_x \log p_\sigma$) of **one** target distribution, and correctness relies on the chain **mixing** to its stationary distribution as $k \rightarrow \infty$, which can be slow when p is multimodal or has low-density barriers. The reverse SDE instead exploits the entire **time-indexed family** of scores $\{\nabla_x \log p_t(x)\}_{t \in [0,1]}$ – a continuum of noise levels interpolating from the data distribution $p_0 \approx p_{\text{data}}$ to the tractable prior $p_1 \approx \mathcal{N}(0, I)$ – **together with the known forward drift** $f(x, t) = -\frac{1}{2}\beta(t)x$. Because the reverse dynamics

$$dx = (f(x, t) - g(t)^2 \nabla_x \log p_t(x))dt + g(t)d\bar{B}_t, \quad g(t) = \sqrt{\beta(t)},$$

exactly inverts the forward diffusion, integrating it from $t = 1$ to $t = 0$ **transports** the prior onto the data manifold along the prescribed reverse trajectory in finite time, with no need to equilibrate to any stationary distribution.

(b) This richer information – scores at **all** noise levels rather than one – is built directly into both training losses:

- **Song21 Eq. (7):** the (denoising) score-matching objective is an **expectation/integral over the whole continuum** $t \in [0, 1]$,

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1]} \left[\lambda(t) \mathbb{E}_{x_0} \mathbb{E}_{x_t | x_0} \left[\left\| s_\theta(x_t, t) - \nabla_{x_t} \log p_{0t}(x_t | x_0) \right\|^2 \right] \right],$$

with a per-level weighting $\lambda(t)$. The single time-conditioned network $s_\theta(x, t)$ is thus trained to match $\nabla_x \log p_t(x)$ **simultaneously at every noise level**, exactly the family the reverse SDE consumes.

- **DDPM Eq. (14):** the simplified loss is a **sum/expectation over all discrete timesteps** $t \in \{1, \dots, T\}$,

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \varepsilon} \left[\left\| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t) \right\|^2 \right],$$

and the one network $\varepsilon_\theta(x_t, t)$ is **conditioned on** t . Since ε_θ is equivalent to a (rescaled) score $\nabla_{x_t} \log q(x_t | x_0)$ at level t (via $\nabla_{x_t} \log q(x_t | x_0) = -\varepsilon / \sqrt{1 - \bar{\alpha}_t}$), conditioning on t and averaging over all t makes it learn the **entire family of time-marginals** $q(x_t | x_0)$, not a single fixed score – precisely the extra information that lets the reverse process denoise step-by-step instead of mixing.

3.7

Classifier guidance steers an unconditional diffusion model toward a target class y by adding the gradient of a noise-aware classifier to the score. By Bayes' rule, at every noise level t ,

$$p_t(x|y) = \frac{p_t(x)p_t(y|x)}{p_t(y)} \propto p_t(x)p_t(y|x) \quad (\text{Bayes' rule; } p_t(y) \text{ const. in } x).$$

Taking $\nabla_x \log$ of both sides and using $\nabla_x \log p_t(y) = 0$ (the marginal $p_t(y)$ does not depend on x),

$$\begin{aligned} \nabla_x \log p_t(x|y) &= \nabla_x \log p_t(x) + \nabla_x \log p_t(y|x) - \underbrace{\nabla_x \log p_t(y)}_{=0} \\ &= \nabla_x \log p_t(x) + \nabla_x \log p_t(y|x). \end{aligned}$$

So the conditional score is the **unconditional** score $\nabla_x \log p_t(x) \approx s_\theta(x, t)$ plus the gradient of a **time-dependent classifier** log-likelihood $\nabla_x \log p_t(y|x_t)$. One therefore trains two models separately: an unconditional diffusion model for s_θ , and a classifier $p_t(y|x_t)$ on noised inputs x_t (it must be noise-level aware, since x_t becomes progressively noisier as t grows). At sampling time one substitutes $\nabla_x \log p_t(x|y) = s_\theta(x, t) + \nabla_x \log p_t(y|x_t)$ into the conditional reverse SDE, so the classifier gradient nudges each reverse step toward regions it assigns high probability to class y ; a guidance scale w multiplying $\nabla_x \log p_t(y|x_t)$ sharpens this push, trading sample diversity for class fidelity.

(Optionally, the **classifier-free** variant avoids training a separate classifier: one jointly learns a conditional and unconditional noise model $\varepsilon_\theta(x, t, y)$ and $\varepsilon_\theta(x, t, \emptyset)$ via conditioning dropout, and guides with $\varepsilon_\theta(x, t, \emptyset) + w(\varepsilon_\theta(x, t, y) - \varepsilon_\theta(x, t, \emptyset))$. Since $\varepsilon_\theta \approx -\sigma(t)\nabla_x \log p_t$, the identity above gives $\varepsilon_\theta(x, t, y) - \varepsilon_\theta(x, t, \emptyset) = -\sigma(t)\nabla_x \log p_t(y|x_t)$, i.e. the difference is exactly the classifier-gradient term scaled by $-\sigma(t)$, so $w = 0$ recovers unconditional and $w = 1$ recovers conditional sampling.)

4

4.1

We minimize over all measurable $v_\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$. Write $Y := X_1 - X_0$. By the tower property (law of total expectation), conditioning on X_t inside the expectation,

$$L_{\text{RF}}(\theta) = \int_0^1 \mathbb{E}[\|Y - v_\theta(X_t, t)\|^2] dt = \int_0^1 \mathbb{E}_{X_t} \left[\underbrace{\mathbb{E}[\|Y - v_\theta(X_t, t)\|^2 | X_t]}_{R(X_t, t)} \right] dt.$$

The integrand $R(x, t) := \mathbb{E}[\|Y - v_\theta(x, t)\|^2 | X_t = x]$ is nonnegative, and for each fixed (x, t) the only dependence on θ is through the single vector $v := v_\theta(x, t) \in \mathbb{R}^d$. Hence integration over t and expectation over X_t are monotone (integration of a nonnegative integrand preserves the pointwise order): if a measurable map v^* minimizes $R(x, t)$ for almost every (x, t) w.r.t. the law of (X_t, t) , then it minimizes L_{RF} . So it suffices to solve, for each fixed (x, t) , the static problem

$$\min_{v \in \mathbb{R}^d} g(v), \quad g(v) := \mathbb{E}[\|Y - v\|^2 \mid X_t = x].$$

Pointwise minimizer (L2 projection / conditional mean). Let $m := \mathbb{E}[Y \mid X_t = x]$, which exists since $Y = X_1 - X_0$ is integrable (assume finite second moments so R is finite). Insert and subtract m and expand the square using $\|a - b\|^2 = \|a\|^2 - 2a^T b + \|b\|^2$:

$$\begin{aligned} \|Y - v\|^2 &= \|(Y - m) + (m - v)\|^2 \\ &= \|Y - m\|^2 + 2(Y - m)^T(m - v) + \|m - v\|^2. \end{aligned}$$

Take the conditional expectation $\mathbb{E}[\cdot \mid X_t = x]$. The vector $m - v$ is deterministic given $X_t = x$ (a fixed function of x), so it pulls out of the cross term by linearity:

$$\mathbb{E}[2(Y - m)^T(m - v) \mid X_t = x] = 2 \left(\underbrace{\mathbb{E}[Y \mid X_t = x]}_{=m} - m \right)^T (m - v) = 0,$$

which vanishes precisely by the definition of m as the conditional mean. Therefore the bias-variance decomposition holds:

$$g(v) = \mathbb{E}[\|Y - v\|^2 \mid X_t = x] = \underbrace{\mathbb{E}[\|Y - m\|^2 \mid X_t = x]}_{\text{conditional variance, indep. of } v} + \|m - v\|^2.$$

The first term does not depend on v and the second satisfies $\|m - v\|^2 \geq 0$ with equality iff $v = m$. Hence $g(v) \geq g(m)$ for all $v \in \mathbb{R}^d$, and the unique minimizer is $v = m = \mathbb{E}[Y \mid X_t = x]$.

First-order check (gradient). Equivalently, g is strictly convex (Hessian $\nabla^2 g = 2I \succ 0$), so its unique stationary point is the global minimizer. Differentiating $g(v) = \mathbb{E}[\|Y\|^2 \mid X_t = x] - 2v^T \mathbb{E}[Y \mid X_t = x] + \|v\|^2$ and setting $\nabla_v g = 0$:

$$\nabla_v g(v) = -2\mathbb{E}[Y \mid X_t = x] + 2v = 0 \quad \Rightarrow \quad v = \mathbb{E}[Y \mid X_t = x] = m,$$

confirming the same answer.

Conclusion. Define $v^*(x, t) := \mathbb{E}[X_1 - X_0 \mid X_t = x]$. This map is measurable in (x, t) (it is a regular conditional expectation), so it is an admissible choice of v_θ . By the argument above it minimizes $R(x, t)$ for every (x, t) , hence pointwise minimizes the integrand of L_{RF} ; since integration over t and expectation over X_t preserve this pointwise optimality,

$$v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x] \in \operatorname{argmin}_{v_\theta} L_{\text{RF}}(\theta).$$

The minimizer is unique up to changes on a set of (x, t) of measure zero under the law of (X_t, t) , since the pointwise minimizer is unique wherever the conditional law is defined.

4.2

Part A. The rectified-flow interpolation is $X_t = (1 - t)X_0 + tX_1$, and the optimal velocity field is the conditional expectation of the instantaneous velocity $\dot{X}_t = X_1 - X_0$ given the current state (the L^2 -optimal velocity is the conditional mean):

$$v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x].$$

From $X_t = (1 - t)X_0 + tX_1$ we solve $X_1 = \frac{X_t - (1-t)X_0}{t}$, hence

$$X_1 - X_0 = \frac{X_t - (1-t)X_0}{t} - X_0 = \frac{X_t - (1-t)X_0 - tX_0}{t} = \frac{X_t - X_0}{t}.$$

Taking $\mathbb{E}[\cdot \mid X_t = x]$ (and using $\mathbb{E}[X_t \mid X_t = x] = x$ by measurability) gives

$$v^*(x, t) = \mathbb{E}\left[\frac{X_t - X_0}{t} \mid X_t = x\right] = \frac{x - \mathbb{E}[X_0 \mid X_t = x]}{t}.$$

Part B. Condition on X_1 . The only remaining randomness is $X_0 \sim \mathcal{N}(0, I)$, and $X_t = (1 - t)X_0 + tX_1$ is an affine function of X_0 , so

$$X_t \mid X_1 \sim \mathcal{N}(tX_1, (1 - t)^2 I) \quad (\text{affine map of a Gaussian}).$$

Writing π_1 for the law of X_1 , the marginal density of X_t is the Gaussian-smoothed density

$$p_t(x) = \int \pi_1(x_1) \mathcal{N}(x; tx_1, (1 - t)^2 I) dx_1.$$

Differentiate under the integral (dominated convergence justifies the interchange, the kernel being smooth and integrable). Using $\nabla_x \mathcal{N}(x; tx_1, (1 - t)^2 I) = -\left(\frac{1}{(1 - t)^2}\right)(x - tx_1) \mathcal{N}(x; tx_1, (1 - t)^2 I)$,

$$\begin{aligned} \nabla_x p_t(x) &= \int \pi_1(x_1) \left(-\frac{1}{(1 - t)^2}\right)(x - tx_1) \mathcal{N}(x; tx_1, (1 - t)^2 I) dx_1 \\ &= -\frac{1}{(1 - t)^2} \int (x - tx_1) \pi_1(x_1) \mathcal{N}(x; tx_1, (1 - t)^2 I) dx_1. \end{aligned}$$

Dividing by $p_{t(x)}$ and recognizing that $\pi_1(x_1) \mathcal{N}(x; tx_1, (1 - t)^2 I) / p_{t(x)} = p(x_1 \mid X_t = x)$ is the posterior over X_1 (Bayes' rule), the integral becomes a conditional expectation:

$$\nabla_x \log p_t(x) = \frac{\nabla_x p_{t(x)}}{p_{t(x)}} = -\frac{1}{(1 - t)^2} (x - t\mathbb{E}[X_1 \mid X_t = x]).$$

This is exactly Tweedie's formula (Problem 2.5) applied to the kernel $X_t \mid X_1 \sim \mathcal{N}(tX_1, (1 - t)^2 I)$. Now solve the interpolation for X_0 : from $X_t = (1 - t)X_0 + tX_1$ we get $X_0 = \frac{X_t - tX_1}{1 - t}$, so taking $\mathbb{E}[\cdot \mid X_t = x]$,

$$\mathbb{E}[X_0 \mid X_t = x] = \frac{x - t\mathbb{E}[X_1 \mid X_t = x]}{1 - t}.$$

Comparing with the score expression above, $x - t\mathbb{E}[X_1 \mid X_t = x] = -(1 - t)^2 \nabla_x \log p_{t(x)}$, hence

$$\mathbb{E}[X_0 \mid X_t = x] = \frac{-(1 - t)^2 \nabla_x \log p_{t(x)}}{1 - t} = -(1 - t) \nabla_x \log p_{t(x)}.$$

Substituting into Part A,

$$v^*(x, t) = \frac{x - \mathbb{E}[X_0 \mid X_t = x]}{t} = \frac{x + (1 - t) \nabla_x \log p_{t(x)}}{t}.$$

Part C. The optimal rectified-flow velocity is an **affine** function of the score: $v^*(x, t) = \frac{x}{t} + \left(\frac{1 - t}{t}\right) \nabla_x \log p_{t(x)}$, with coefficients depending only on t . Thus knowing v^* is equivalent to knowing the score $\nabla_x \log p_t$ (each determines the other), so rectified flow and score-based diffusion encode the **same** information about the data. Their deterministic generative dynamics are the same probability-flow ODE written in different coordinates, coinciding up to the time reparameterization / deterministic change of variables that maps the linear-interpolation schedule to the VP-SDE schedule.

4.3

Throughout, $v_\theta(x, t)$ is the learned velocity field of a rectified flow whose probability-flow ODE is $dX_t = v_\theta(X_t, t)dt$, and (X_0, X_1) are the endpoint pairs (noise, data) under some coupling, with linear interpolant $X_t = (1 - t)X_0 + tX_1$ so that the **target** velocity along an interpolation line is the constant $X_1 - X_0$ (definition of rectified flow). Write Φ^s for the time- s flow map of this ODE.

(1) Geometric meaning of the extreme values of S .

Recall

$$S = 1 - \frac{\mathbb{E}\left[\int_0^1 \|v_\theta(X_t, t) - (X_1 - X_0)\|^2 dt\right]}{\mathbb{E}[\|X_1 - X_0\|^2]}.$$

$S = 1$: the numerator vanishes, so $v_\theta(X_t, t) = X_1 - X_0$ for (almost) all t along every trajectory; the velocity is **constant in time**, hence each path is a straight, constant-speed line, and a single Euler step of size $\Delta t = 1$, namely $X_0 \mapsto X_0 + v_\theta(X_0, 0) = X_0 + (X_1 - X_0) = X_1$, integrates the ODE **exactly** (no discretization error).

$S = 0$: the mean-squared deviation of v_θ from the straight displacement equals the displacement energy $\mathbb{E}[\|X_1 - X_0\|^2]$ itself, i.e. the velocity field is “wrong” on the scale of the data, so the flow is maximally curved / as far from straight as the data scale allows and a one-step Euler integration is useless.

(2) Why re-pairing (reflow) straightens trajectories.

The marginal velocity that the regression actually fits is the conditional expectation of the per-pair target given the current state (tower property / least-squares optimum of the flow-matching loss):

$$v^*(x, t) = \mathbb{E}[X_1 - X_0 \mid X_t = x].$$

Under the **original** independent coupling $\pi_0 \times \pi_1$, many interpolation lines pass through the same intermediate point x with **different** displacements $X_1 - X_0$; the lines cross, so the conditioning above averages several distinct directions and $v^*(x, t)$ varies with t along a realized path $\Rightarrow$ curved trajectories ($S < 1$). After the first training the ODE is deterministic, so it induces the coupling $(X_0, \Phi^1(X_0))$ in which each noise sample maps to a **single** output (definition of a deterministic transport map). Repairing on $(X_0, \Phi^1(X_0))$ gives a coupling whose interpolation lines do **not** cross transversally (a deterministic, monotone-like transport has non-intersecting characteristics), so at each x the conditional expectation $\mathbb{E}[X_1 - X_0 \mid X_t = x]$ concentrates on essentially one displacement direction. Hence v^* is more nearly constant along each line, the paths are straighter, and S increases toward 1. Crucially, reflow preserves the marginals ($\Phi^1(X_0) \sim \pi_1$ by construction), so straightening does not change the generated distribution in the ideal case.

(3) Cost of one reflow round, and why to limit rounds.

One reflow round costs roughly **(one generation pass) + (one full retraining run)**:

(a) **Data generation**: to build the new pairs one must simulate the learned ODE $X_1 = \Phi^1(X_0)$ for $\sim N$ samples. With a K -step numerical integrator (e.g. $K \approx 100$ Euler steps) this is $\approx K \cdot N$ network evaluations, i.e. $\tilde{O}(KN)$ forward passes; this dominates because the original training never needed full trajectory simulation (its targets are the cheap analytic interpolants $X_1 - X_0$).

(b) **Retraining:** a complete optimization run of the flow-matching loss on the N regenerated pairs, comparable in cost to the original training.

So each round adds a generation pass on top of a fresh training run, several times the cost of plain training. One limits the number of rounds for two reasons: the per-round cost above is large, and (more importantly) the regression **targets** are produced by the imperfect previous model rather than by ground-truth data. Each round therefore feeds simulation error and the model's own bias back in as supervision, so approximation error **compounds** across rounds; after one or two rounds the straightness gains saturate while sample-quality degradation from accumulated error can grow, making further rounds a poor trade.

4.4

We compare DDPM (Ho et al. 2020) against Rectified Flow (RF; Liu et al. 2022) along four axes. Throughout, RF couples a data sample $X_1 \sim p_{\text{data}}$ with noise $X_0 \sim \mathcal{N}(0, I)$, defines the interpolation $X_t = (1 - t)X_0 + tX_1$ for $t \in [0, 1]$, and learns a velocity field $v_\theta(x, t)$.

(1) **Training objective.** DDPM regresses the injected Gaussian noise: with $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$ and $\varepsilon \sim \mathcal{N}(0, I)$, the (simplified) loss is

$$L_{\text{DDPM}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2],$$

so the regression **target** ε is **t -dependent** and (equivalently, by Tweedie/the score identity $\nabla_{x_t} \log q(x_t|x_0) = -\varepsilon/\sqrt{1 - \bar{\alpha}_t}$) amounts to learning the score. RF instead regresses the **constant displacement** $X_1 - X_0$ of each pair:

$$L_{\text{RF}} = \mathbb{E}_{t, X_0, X_1} [\|(X_1 - X_0) - v_\theta(X_t, t)\|^2],$$

whose target $\partial_t X_t = X_1 - X_0$ is **time-independent** along a given pair's straight interpolant.

(2) **Inference cost.** DDPM's reverse path is a **curved, stochastic** trajectory, so an accurate solve needs a fine time discretization (many network evaluations, classically $T \sim 10^3$, hundreds even with fast samplers). RF integrates the deterministic ODE $dx = v_\theta(x, t)dt$, whose trajectories are (by construction, and increasingly so after **reflow**) close to straight, so few Euler steps suffice. In the ideal straightened limit ($S = 1$: velocity exactly constant along each trajectory) a **single** Euler step with $\Delta t = 1$,

$$\hat{x}_1 = x_0 + 1 \cdot v_\theta(x_0, 0),$$

is **exact**, since for constant v the Euler update incurs zero truncation error.

(3) **Trajectory geometry.** DDPM evolves along a **curved SDE** (stochastic, with diffusion term $g(t)d\bar{B}_t$); RF evolves along a **deterministic ODE** whose paths are **ideally straight lines** $x_0 + t(X_1 - X_0)$. The implication is quantitative: local Euler truncation error scales like $(\Delta t)^2 \|\partial_t^2 x\|$, and a straight path has $\partial_t^2 x = 0$. Thus **straighter $\Rightarrow$ smaller per-step discretization error $\Rightarrow$ fewer steps needed** for a target accuracy. (This straightness is exactly what reflow optimizes: it never increases, and generally decreases, transport cost while reducing path curvature.)

(4) **Likelihood evaluation.** DDPM's training loss yields only the **ELBO**, a variational **lower bound** $\log p_\theta(x_0) \geq -L$ (by Jensen, as in 1.2); the gap is a sum of KL terms and is generally nonzero, so DDPM gives a bound, not the exact log-likelihood. RF's deterministic ODE makes it a **continuous normalizing flow**, so it admits an **exact** likelihood via the instantaneous change-of-variables formula,

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla_x \cdot v_\theta(x_t, t) dt,$$

integrating the divergence (trace of the Jacobian) of the velocity along the trajectory. However, this is **not** delivered by the training objective: it requires solving the ODE and estimating $\text{tr}(\nabla_x v_\theta)$ (e.g. via a Hutchinson estimator), so exact likelihood is available **in principle**, not for free.

5

All code for this part is implemented in `diffusion/vp.py` (submitted to Gradescope as `VP.py`); the DSM training loss is in `scripts/train_vp.py` and sampling drivers are in `scripts/sample.py`. All autograder tests in `tests/test_vp.py` pass. Figures marked [run] are produced by the Colab notebook on an A100 and should be pasted in from `runs/vp/`.

5.A.i (Drift and Diffusion Coefficients)

The general Itô SDE is written

$$dx = f(x, t)dt + g(t)dB_t,$$

where $f(x, t)$ is the drift coefficient (the deterministic per-unit-time increment) and $g(t)$ is the diffusion coefficient (the scaling of the Brownian increment). Comparing term-by-term with the VP-SDE

$$dx = -\frac{1}{2}\beta(t)xdt + \sqrt{\beta(t)}dB_t,$$

we read off the dt -coefficient as the drift and the dB_t -coefficient as the diffusion:

$$f(x, t) = -\frac{1}{2}\beta(t)x, \quad g(t) = \sqrt{\beta(t)}.$$

Note f is linear in x (giving an affine, Gaussian-preserving drift) and g depends only on t , not on x (additive noise), which is exactly the structure that yields the closed-form Gaussian perturbation kernel $x_t = c(t)x_0 + \sigma(t)\varepsilon$.

5.A.ii ($\beta(t)$ and $c(t)$)

For the VP-SDE with the linear noise schedule (Song21 Eqs. 32–33),

$$\beta(t) = \beta_{\min} + (\beta_{\max} - \beta_{\min})t, \quad t \in [0, 1].$$

The mean-decay factor of the perturbation kernel is $c(t) = \exp\left(-\frac{1}{2} \int_0^t \beta(s) ds\right)$. Evaluating the integral (linear integrand, with antiderivative $\beta_{\min}s + \frac{1}{2}(\beta_{\max} - \beta_{\min})s^2$):

$$\int_0^t \beta(s) ds = \int_0^t [\beta_{\min} + (\beta_{\max} - \beta_{\min})s] ds = \beta_{\min}t + \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2.$$

Hence

$$c(t) = \exp\left(-\frac{1}{2}\left(\beta_{\min}t + \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2\right)\right).$$

This $c(t)$ is the mean-decay factor in the (Gaussian) perturbation kernel

$$x_t = c(t)x_0 + \sigma(t)\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad \sigma(t)^2 = 1 - c(t)^2,$$

where $\sigma(t)^2 = 1 - c(t)^2$ enforces the variance-preserving property: with $c(t)^2 + \sigma(t)^2 = 1$, a unit-variance x_0 gives marginal variance $c(t)^2 \cdot 1 + \sigma(t)^2 = 1$ for all t .

Verification at $t = 0$: the integral vanishes ($\int_0^0 \beta(s) ds = 0$), so $c(0) = \exp(0) = 1$, giving $x_0 = c(0)x_0 + \sigma(0)\varepsilon = x_0$ with $\sigma(0)^2 = 1 - 1 = 0$ (no noise at $t = 0$), as required.

5.A.iii (Implementation)

Implemented in `VP.py`: `beta`, `c`, `sigma`, `drift`, `diffusion`, and `marginal`. The score network is trained with the likelihood-weighted DSM loss (Song21 Eq. 7); with target score $-\varepsilon/\sigma(t)$ and weight $\lambda(t) = \sigma(t)^2$ this reduces to the clean form $\mathbb{E}[\|\sigma(t)s_\theta(x_t, t) + \varepsilon\|^2]$.

5.B (Samplers)

`Euler_Maruyama_sampler` integrates the reverse VP-SDE $dx = [-\frac{1}{2}\beta(t)x - \beta(t)\nabla_x \log p_t(x)]dt + \sqrt{\beta(t)}d\bar{B}_t$ backward from $t = 1$ to $t \approx 0$; at each step $x \leftarrow x + (\frac{1}{2}\beta(t)x + \beta(t)s_\theta)\Delta t + \sqrt{\beta(t)\Delta t}z$, returning the final mean. `predictor_corrector_sampler` (Algorithm 5 of Song21) alternates one EM predictor step with `n_corrector` annealed-Langevin corrector steps with step size $2(r\|z\|/\|s_\theta\|)^2$ (r = target SNR).

5.C.i (Dataset Visualization)

FashionMNIST — 64 training images (28x28 grayscale, 10 classes)



FashionMNIST contains 60{,}000 training and 10{,}000 test images. Each image is a single-channel $1 \times 28 \times 28$ grayscale tensor (we normalize to $[-1, 1]$). There are 10 balanced classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. The prior is dominated by centered, roughly symmetric garment silhouettes on a black background.

5.C.ii (Training Curves)

[run] Paste runs/vp/ loss curve here (log-scale y). Settings: 50 epochs, learning rate 10^{-4} , batch size 128, $[\beta_{\min}, \beta_{\max}] = [0.01, 5.0]$, Adam, early-stopping patience 10. The weighted DSM loss decreases quickly over the first few epochs and then plateaus; the best checkpoint (lowest validation loss) is kept.

5.C.iii (EM Samples)

[run] Paste the 8×8 EM grid from sample.py --method em --checkpoint runs/vp/best.pt --num_steps 1000.

5.C.iv (PC Samples)

[run] Paste two 8×8 PC grids, e.g. --n_corrector 1 and --n_corrector 3 (sample.py --method pc ...).

5.C.v (Qualitative Discussion)

EM alone leaves visible high-frequency noise at moderate step counts, since it only follows the first-order reverse-SDE discretization. The PC sampler interleaves Langevin corrector steps that pull each iterate back onto the data manifold, yielding cleaner, more globally coherent garments; increasing the corrector count from 1 to 3 sharpens edges further at extra compute cost (too many steps eventually oversmooths fine texture). Compared with the dataset plot (5.C.i), the model captures the dominant low-frequency structure well — overall class silhouettes such as trousers, bags, and ankle boots — but struggles with fine high-frequency detail (logos, sandal straps, fabric texture) and with visually ambiguous classes (Shirt vs. Coat vs. Pullover).

6

Rectified-flow forward process, loss, Euler sampler, and reflow are implemented in diffusion/rectflow.py training/reflow drivers are in scripts/train_rectflow.py, KID evaluation in scripts/eval_kid.py. All tests in tests/test_rectflow.py pass.

6.A (Forward Process and Training Loss)

The rectified-flow forward process draws $x_0 \sim \mathcal{N}(0, I)$, forms $x_t = (1 - t)x_0 + tx_1$ with $t \sim \text{Uniform}(0, 1)$, and regresses the (time-independent) velocity $x_1 - x_0$: $\mathcal{L}_{\text{RF}} = \mathbb{E}\|x_1 - x_0 - v_\theta(x_t, t)\|^2$. [run] Paste the combined (RF + VP) training-loss curve, log-scale. **Do the loss scales compare directly?** No. The VP/DDPM model regresses the Gaussian noise ϵ under a $\sigma(t)^2$ weighting, whereas rectified flow regresses the velocity $x_1 - x_0$ under a flat (unweighted) MSE. The two targets have different distributions and the objectives use different weightings, so the *absolute* loss magnitudes are not comparable — only the trend (monotone decrease and convergence) can be read across the two curves.

6.B (Euler Sampler and Full Comparison)

[run] Fill the KID table (mean $\pm$ std, 1k samples via torch-fidelity) and paste the 8×8 grids for each (method, steps). All grids share the same initial noise x_0 per cell.

Steps	Flow Matching	DDIM	DDPM EM
1	[run]	[run]	—
5	[run]	[run]	—
10	[run]	[run]	—

Steps	Flow Matching	DDIM	DDPM EM
50	[run]	[run]	—
100	[run]	[run]	—
200	[run]	[run]	—
1000	[run]	[run]	baseline

Expected qualitative findings (confirm against your runs):

1. **First recognizable samples.** Flow matching is recognizable at the fewest steps (~ 5 – 10), DDIM a bit later (~ 10 – 50); DDPM EM, being stochastic and first-order, needs the most ($\sim$ hundreds) before the noise floor is low enough.
2. **Steps to match 1000-step DDPM EM.** Both deterministic samplers reach baseline quality far sooner — typically ~ 50 – 100 steps for flow matching and DDIM — while DDPM EM needs its full 1000.
3. **At 100 steps.** Flow matching and DDIM are close to each other and to 1000-step DDPM EM. Since the *same* model can be sampled to baseline quality with $10 \times$ fewer steps using a better (deterministic) sampler, the low-step bottleneck is the *sampler / discretization*, not the model.
4. **Best vs. fast.** For best quality regardless of speed, pick the lowest-KID cell (1000-step DDPM EM, or DDIM/flow at 100–200). For fast generation with acceptable quality, use flow matching at a handful of steps (or a single step after reflow, 6.C).

6.C (One-Step Generation after Reflow)

[run] Paste the one-step ($\Delta t = 1$) reflow grid and fill the row below.

Method	Steps	Sample grid	FID	Time (s/64)
Rect. Flow — Reflow	1	[run]	[run]	[run]

Reflow re-pairs each noise sample with its ODE output and retrains, straightening the trajectories. As a result the one-step reflow FID is dramatically better than one-step rectified flow *without* reflow (whose single Euler step badly under-integrates a curved path). It remains somewhat worse than 1000-step DDPM EM, but comes close while using $\sim 1000 \times$ fewer function evaluations — a large quality-for-speed win.

6.D (Side-by-Side Qualitative Grid)

[run] Paste the 4×8 grid produced by `sample.py --method all ... --seed 42` (rows: DDPM EM 1000, RF 100, RF 1, Reflow 1; same 8 seeds across all rows). Because the rectified-flow rows are deterministic ODE integrations of the *same* velocity field family, a fixed initial noise tends to map to semantically similar outputs (same class/shape) across RF-100, RF-1, and Reflow-1, differing mainly in sharpness. DDPM EM is stochastic, so even with the same initial x_0 its output can drift to a different instance. The flow models generally give cleaner edges, while DDPM may show different texture/artifact patterns.

7

Part 7 uses OpenAI's guided-diffusion codebase on 256×256 ImageNet models; runs are done in Colab on an A100 (5–10 min each). Below are the commands and expected observations; paste each 256×2048 figure from the corresponding .npz via `scripts/guided_diffusion_experiments.py`.

```
MODEL_FLAGS="--attention_resolutions 32,16,8 --class_cond False --diffusion_steps 1000 \
```

```
--image_size 256 --learn_sigma True --noise_schedule linear --num_channels 256 \
--num_head_channels 64 --num_res_blocks 2 --resblock_updown True --use_fp16 True \
--use_scale_shift_norm True"
SAMPLE_FLAGS="--batch_size 8 --num_samples 8 --timestep_respadding 250"
```

7.1 (Unconditional Image Generation)

```
python scripts/image_sample.py $MODEL_FLAGS \
```

--model_path models/256x256_diffusion_uncond.pt \$SAMPLE_FLAGS [run] *Paste the 256 × 2048 figure (8 unconditional samples).* Comments: samples are diverse, globally coherent natural images; at 256 × 256 they show realistic large-scale structure with occasional implausible fine details, typical of an unconditional model with no class signal.

7.2 (Progressive Generation)

Use the progressive sampler (e.g. p_sample_loop_progressive) and save 8 evenly-spaced intermediates from $t = T$ (noise) to $t = 0$ (final). [run] *Paste the 8-column timeline.* Comments: early columns are pure noise; coarse layout/colors emerge first (low frequencies), then progressively finer edges and texture — the reverse process is coarse-to-fine.

7.3 (Noise Interpolation)

Fix the seed so only the initial latent varies; pick $z_0, z_7 \sim \mathcal{N}(0, I)$ whose samples differ strongly, then sample from $z_i = (1 - \frac{i}{7})z_0 + (\frac{i}{7})z_7$, $i = 0, \dots, 7$. [run] *Paste the 8-column interpolation.* Comments: the decoded images vary smoothly between the two endpoints, indicating the model's latent→image map is continuous (semantically meaningful interpolation rather than abrupt jumps).

7.4 (Conditional Image Generation)

```
CLASSIFIER_FLAGS="--classifier_scale 1.0 --classifier_path
models/256x256_classifier.pt \
--classifier_attention_resolutions 32,16,8 --classifier_depth 2 --classifier_width
128 \
--classifier_pool attention --classifier_resblock_updown True \
--classifier_use_scale_shift_norm True"
python scripts/classifier_sample.py $MODEL_FLAGS $CLASSIFIER_FLAGS \
```

--model_path models/256x256_diffusion_uncond.pt \$SAMPLE_FLAGS [run] *Paste 8 class-conditional samples (8 random ImageNet classes).* Comments: each image now clearly depicts its target class; classifier guidance trades a little diversity for much stronger class adherence relative to 7.1.

7.5 (Classifier Scale Sweep)

Sweep --classifier_scale over 8 increasing values (e.g. 0.5, 1, 2, 4, 6, 8, 10, 12) with the same seed/initial state; produce 2 rows × 8 columns. [run] *Paste the 512 × 2048 figure.* Comments: small scales give diverse but weakly class-aligned samples; increasing the scale sharpens class identity and fidelity but reduces diversity and eventually introduces saturation/ over-guidance artifacts — the classic fidelity–diversity trade-off.